Geometry Modeling: Lab 1

Spring 2025

1 Problem

You must accomplish the following tasks in the given order:

1. Write the function `myMesh::readFile` to read a mesh in `.obj` format into a half-edge data structure.
2. Implement the functions to compute normal vectors for faces and vertices, in exactly this order. The classes `myFace` and `myVertex` provide functions to compute normals for individual elements, i.e. per face or per vertex. Once this is done, in class `myMesh` you need to identify the methods responsible for computing normals for all elements by calling the dedicated per-element method inside a loop. Note that function such as cross product are already implemented.
3. Once the normals are computed, you must ask for further instructions on how to implement the silhouette.
4. Finally, implement a function to triangulate a mesh by first implementing a function capable of triangulating a single face, and then applying it in a loop to all non-triangular faces. You may use either the ear-clipping algorithm or a method that adds a new vertex at the center of a face and splits the face into triangles using this new vertex.

2 Where to start? Reading a Mesh into a Half-Edge Structure

start by studying the available classes:

- `myVertex` : Stores vertex information.
- `myHalfedge` : Stores halfedge information.
- `myFace` : Stores face information.
- `myMesh` : Stores mesh information.

Functions to implement:

- `void readFile(std::string filename)` in the class `myMesh` to read a `.obj` file and store the data in the halfedge structure.

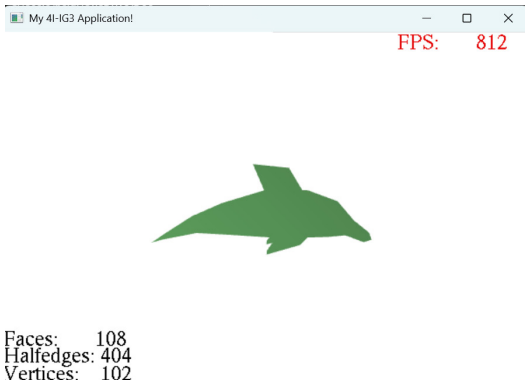
For more details on the `.obj` format, see: `http://www.cs.clemson.edu/~dhouse/courses/405/docs/brief-obj-file-format.html`.

Notice that the code already defines an `std::map` container, which should be used to resolve twin half-edges. Please read the documentation for `std::map`: `https://en.cppreference.com/w/cpp/container/map.html` and get yourself familiar with the provided code, first!

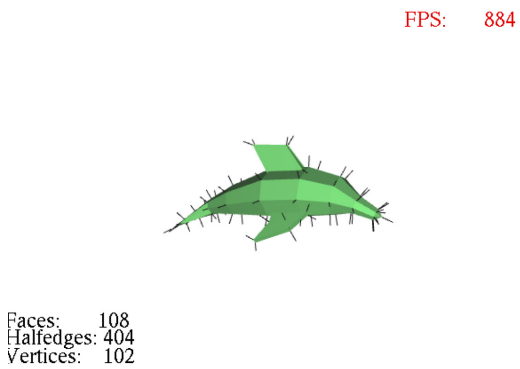
3 Recommended Order for Completing the Lab

1. `myMesh::readFile()` : Allows loading meshes from OBJ file format data.
2. `computeNormal()` for faces and vertices.
3. Filling in the blanks in the `main.cpp` in the if section corresponding to drawing the silhouette
4. `myMesh::triangulate()`

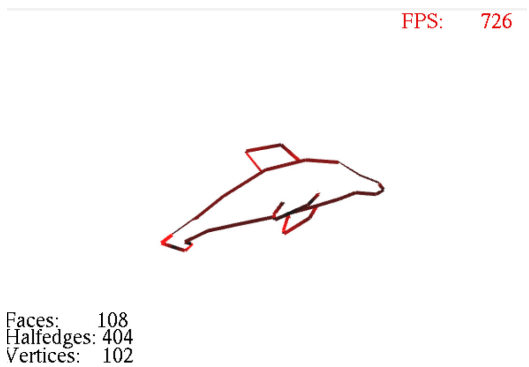
4 Expected results



(a)



(b)



(c)

Figure 1: Dolphin mesh visualizations: (a) rendered mesh showing the dolphin with the expected number of geometric primitives; (b) dolphin mesh with vertex normals displayed; (c) silhouette of the dolphin mesh.